Marwadi University
Marwadi Chandarana Group

FACULTY OF TECHNOLOGY | DAA RESEARCH ASSIGNMENT

Algorithm-Based Resource Allocation in Cloud Computing

A Cost Optimization Study

RESEARCH FOCUS

Cost-aware placement of computing workloads under resource and service constraints.

Submitted by	SontiReddy AshokReddy
Enrollment Number	92460118649
Course / Department	CSE AI&ML
Institution	MARWADI UNIVERSITY
Submitted to	MR. ABHISHEK CHAUHAN
Academic Year	2026-2027

Prepared as a literature-based DAA study with a worked illustrative example.

Abstract

Cloud platforms allow users to request computing resources when needed, but the flexibility of the cloud does not automatically make every allocation economical. If a workload receives more CPU or memory than it needs, the provider or customer may pay for unused capacity. If too little capacity is assigned, response time can increase and service-level objectives may be missed. Resource allocation is therefore an algorithmic decision problem in which cost, capacity, workload duration, and service requirements must be considered together [1-3].

This paper reviews algorithmic approaches to cloud resource allocation, with emphasis on cost-aware allocation of virtual machines (VMs). It outlines the problem, presents a simplified mathematical model, and describes a transparent cheapest-feasible-fit procedure. A small numerical example compares an over-provisioned baseline with a capacity-matched allocation. The example uses hypothetical prices and is intended to demonstrate the calculations; it is not a cloud benchmark or a claim of measured savings. The review also discusses the limits of greedy methods and why production systems need monitoring, fairness controls, and service-level safeguards.

Keywords: cloud computing, resource allocation, virtual machines, cost optimization, task scheduling, greedy algorithm, quality of service.

1. Introduction

Cloud computing provides on-demand access to processing, memory, storage, and network services. In Infrastructure as a Service (IaaS), users can choose virtual machine types and pay according to the service model offered by a provider. At the same time, a cloud data centre must place workloads across physical servers while balancing resource use, performance, energy consumption, and operating cost [1,2].

The difficulty is that workloads differ. A small API service may need little memory but low response time; a batch job may need more CPU and tolerate a longer completion time; a data-processing task may need both memory and storage throughput. A single allocation rule is unlikely to be best for every workload. Resource-management research therefore studies provisioning, placement, scheduling, consolidation, and load balancing as related but distinct decisions [3,4].

This paper narrows the problem to cost-aware assignment of independent jobs to eligible VM types. The goal is to show how a clear allocation rule can reduce avoidable over-provisioning while respecting the minimum resources needed by each job.

1.1 Problem statement

Given a set of workloads, each with a resource demand and estimated runtime, select a suitable VM type for every workload to minimize estimated compute cost, subject to resource eligibility and service requirements. The problem becomes harder when workloads overlap, resource demands change, prices differ by region or time, or jobs have deadlines.

1.2 Objectives

- Explain the cloud resource-allocation problem in DAA terms.
- Compare exact optimization, greedy heuristics, metaheuristics, and learning-based approaches.
- Present a cost model and a simple, reproducible allocation procedure.
- Identify assumptions, limitations, and appropriate evaluation measures.

2. Research Methodology and Background

This is a narrative literature review based on published work about virtual-machine placement, resource provisioning, scheduling, and cloud resource management. The source set includes foundational survey work and recent reviews identified through publisher and academic-index pages. The purpose is to explain the algorithmic choices and their trade-offs, not to claim a systematic review or an original production experiment. The quantitative example later in the paper is constructed solely for explanation.

2.1 Important terms

Term	Meaning in this study
Workload / job	A unit of computation with a known or estimated CPU, memory, and runtime requirement.
Virtual machine (VM)	A software-defined compute instance with a specified capacity and price.
Provisioning	Choosing how much resource capacity to make available for expected demand.
Placement / allocation	Assigning jobs or VMs to available instances or physical hosts.
SLA / SLO	An agreement or target describing required availability, response time, or completion time.
Over-provisioning	Allocating more capacity than the workload can use, which can increase spend.

2.2 Related research

Mann surveys VM allocation problem models and optimization algorithms, highlighting that energy and monetary cost must be balanced against performance and that comparisons can be misleading when studies use different assumptions [1]. Zhang, Huang, and Wang classify resource-provisioning approaches and show that algorithm selection depends on the objective and the phase of VM placement [2]. Fard and colleagues review resource-allocation mechanisms and identify concerns beyond price, including utilization, load balance, service quality, migrations, and resource availability [3].

Aron and Abraham examine scheduling methods involving heuristics, metaheuristics, and artificial intelligence; their review emphasizes the complexity introduced by heterogeneous, interdependent, and unpredictable workloads [4]. A later review of IaaS scheduling organizes techniques and evaluation approaches while discussing cost, time, utilization, and quality-of-service trade-offs [5]. A 2026 systematic review of AI-driven allocation analyzed 63 selected studies and reported that comparable cost data were available for only a small subset; it also identified simulation-heavy evaluation and limited carbon-aware scheduling as important gaps [6]. These findings support a careful conclusion: no single algorithm is universally cheapest or best.

2.3 Why a cost-only rule is insufficient

An allocation that minimizes a bill but violates a deadline is not a good allocation. Likewise, concentrating every workload on one busy host may save infrastructure cost but create contention or a failure hotspot. A realistic objective should include a clear cost measure together with feasibility constraints for CPU, memory, runtime, availability, and any applicable service-level target.

3. Problem Formulation

Let T be a set of jobs and V be a set of available VM types. Each job i has a demand vector r_i, an estimated runtime d_i, and (where relevant) a deadline or response-time target. Each VM type j has a resource capacity vector R_j and an hourly price p_j. A job can be assigned only to a VM that satisfies its minimum resource requirements.

For the simplified case in which each job is assigned to one VM instance, an estimated compute-cost objective is:

$$\text{Minimize: } C = \Sigma(i \text{ in } T)\ \Sigma(j \text{ in } V)\ x_{ij} \times p_j \times d_i$$

where $x_{ij} = 1$ when job i is assigned to VM type j and 0 otherwise. The model is subject to:

$\Sigma(j \text{ in } V)\ x_{ij} = 1$ for every job i; and $x_{ij} = 0$ whenever VM type j cannot satisfy the minimum resource demand of job i. Additional constraints can represent deadlines, availability, data locality, security, or provider-specific rules.

The formula is intentionally simplified. Real cloud bills can include storage, data transfer, licenses, minimum billing intervals, discounts, reserved capacity, and regional pricing. If multiple jobs share a host, the model must also check the combined CPU and memory demand over time. If runtime is uncertain, a safety margin or probabilistic service constraint may be needed.

3.1 What should be measured?

Metric	Why it matters	Possible measurement
Estimated cost	Primary optimization target in this paper.	Sum of instance rate multiplied by billed duration.
Feasibility	Ensures a job receives enough resources.	Percentage of allocations meeting CPU and memory needs.
SLA/SLO violations	Avoids saving money by degrading service.	Missed deadlines, latency percentiles, or failed jobs.
Utilization	Shows whether capacity is under-used or saturated.	CPU and memory use over the observation period.
Stability / overhead	Frequent rescheduling can itself have a cost.	Migration count, decision time, and startup overhead.

3.2 Decision scope

The example algorithm below is a baseline heuristic: it chooses the lowest-priced VM type that can feasibly run each job. This is easy to implement and explain, but it does not prove global optimality when jobs compete for shared hosts or when consolidation and startup costs matter. For small, well-defined cases, exact integer optimization can be used as a benchmark; for large dynamic cases, a heuristic or hybrid approach may be more practical.

4. Proposed Cost-Aware Allocation Procedure

A practical first step is to filter out VM types that cannot satisfy a workload's minimum requirements, then compare the estimated cost of the remaining choices. The selected VM is the cheapest feasible option, with a policy check for service constraints. A monitoring step should compare the prediction with actual usage and update future decisions.

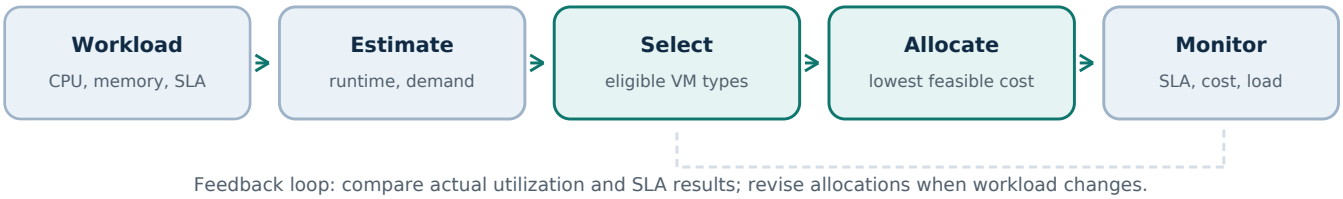


Figure 1. Workflow for a simple cost-aware cloud resource allocation process. The feedback step is necessary because real workloads and runtime estimates can change.

4.1 Pseudocode

```
INPUT: jobs T, VM types V, estimated runtime d[i], hourly price p[j]
FOR each job i in T:
    feasible = empty list
    FOR each VM type j in V:
        IF VM j meets job i resource and policy requirements:
            estimated_cost = p[j] * d[i]
            add (j, estimated_cost) to feasible
    IF feasible is empty:
        mark job i as unallocated and raise a capacity alert
    ELSE:
        choose the feasible VM with minimum estimated_cost
        allocate job i subject to current capacity and SLA checks
        record chosen type, estimated cost, and allocation reason
MONITOR actual resource use and SLA outcomes; revise estimates when needed
```

4.2 Complexity

If there are n jobs and m VM types, the basic scan evaluates each VM type for each job, giving O(nm) time. It stores a small number of candidate choices at a time, so the core scan uses O(m) auxiliary space per job (or O(n) space if the chosen assignment for every job is stored). This complexity excludes the cost of estimating runtimes, querying provider prices, checking live capacity, and rescheduling shared workloads. If the job-placement problem includes interactions between jobs, the independent-job scan is not enough to solve the full packing problem.

4.3 Implementation safeguards

- Reject allocations that fail minimum CPU, memory, or policy requirements.
- Add a configurable safety margin when runtime or demand estimates are uncertain.
- Record why a VM was selected, including price, capacity, and any SLA constraint.
- Re-evaluate allocations only when the likely savings justify migration or restart overhead.

5. Worked Example: Illustrative Cost Comparison

The following example is a hand-calculated demonstration. The dollar prices are fictional values selected to make the arithmetic easy; they are not current prices from AWS, Azure, Google Cloud, or any other provider. Assume a job is assigned its own VM for the duration shown, and the smallest VM satisfying its resource demand is eligible.

Table 1. Illustrative VM types and hourly prices

VM type	CPU capacity	Memory	Illustrative rate / hour
Small	2 vCPU	4 GB	$0.10
Medium	4 vCPU	8 GB	$0.18
Large	8 vCPU	16 GB	$0.32

Table 2. Four independent jobs and the cheapest feasible choice

Job	Demand (vCPU, RAM)	Duration	Chosen VM	Cost calculation	Cost
J1	2 vCPU, 4 GB	3 h	Small	3 x $0.10	$0.30
J2	4 vCPU, 8 GB	2 h	Medium	2 x $0.18	$0.36
J3	6 vCPU, 12 GB	1 h	Large	1 x $0.32	$0.32
J4	1 vCPU, 2 GB	4 h	Small	4 x $0.10	$0.40
Total	-	-	-	-	$1.38

For comparison, suppose a simple baseline assigns the large VM type to every job, regardless of actual need. The estimated cost is:

$$\text{Baseline} = (3 + 2 + 1 + 4)\ \text{hours} \times \$0.32 = \$3.20$$

The cheapest-feasible choices cost $1.38, which is $1.82 lower than this deliberately over-provisioned baseline. Relative to the baseline, the illustrative reduction is (3.20 - 1.38) / 3.20 x 100 = 56.9%. This percentage describes only the small made-up example; it is not a result from a benchmark or evidence that the heuristic will deliver the same reduction in a real cloud.

5.1 What the example does and does not show

- It shows how resource eligibility and hourly price can be combined in a transparent decision rule.
- It illustrates how assigning a large instance to a small workload can raise the bill without adding useful capacity.
- It does not model simultaneous jobs sharing one host, variable demand, hidden fees, setup delays, or strict latency targets.
- A proper experiment would use published workloads or a clearly documented trace, repeat the test, and compare both cost and SLA metrics.

6. Comparison of Algorithmic Approaches

Approach	Main idea	Strength	Main weakness	Typical role
Static / rule-based	Use fixed thresholds or predefined VM choices.	Simple, fast, easy to audit.	May waste resources when demand changes.	Stable or small workloads.
Greedy cheapest-fit	Choose the least expensive feasible option at each decision.	Transparent and low decision overhead.	Can miss a better global assignment.	Baseline, independent jobs, initial filter.
Exact optimization	Formulate constraints and solve a mathematical model.	Can find an optimum for a well-defined model.	May become expensive as the problem grows.	Small/medium planning windows; benchmark.
Metaheuristics	Search for strong solutions through methods such as genetic or swarm-based search.	Can explore complex combinations.	May require tuning; no general optimality guarantee.	Large placement or multi-objective problems.
ML / reinforcement learning	Predict demand or learn allocation policies from data and feedback.	Can adapt to recurring patterns and dynamic states.	Needs representative data, testing, and safeguards.	Dynamic platforms after careful evaluation.

Table 3. High-level comparison synthesized from the reviewed literature [1-6]. The categories overlap: a production allocator can combine a predictive model, a greedy first choice, and an exact optimizer for a small critical subset.

6.1 Discussion

The cheapest-feasible heuristic is a useful starting point because its decision is easy to explain and its computational cost is low. However, the rule makes a local choice for one job at a time. If jobs can share a VM or if placement choices compete for a limited pool of hosts, an individually cheap choice may use capacity that would have supported another job more efficiently. The placement problem then resembles a constrained packing problem, and the order in which workloads are considered can affect the result [1,2].

Exact optimization can serve as a reference when the model is small enough, but not every real scheduling problem can be solved optimally in the time available. Metaheuristics may search a larger set of solutions without proving optimality. Learning-based methods may help predict demand or adjust decisions, but they should be evaluated against simple baselines under the same workload and constraints. A 2026 review reported that only a subset of reviewed studies offered quantifiable cost comparisons and noted the lack of standardized evaluation practices [6]. That is a reason to report assumptions and raw measures rather than quote a universal savings percentage.

6.2 Recommended evaluation design

- Use at least one simple baseline such as fixed-size provisioning and one stronger method such as best-fit or an optimizer.
- Keep VM prices, job traces, runtime estimates, and service constraints identical across methods.
- Report total cost alongside completion time, SLA violations, utilization, decision latency, and failures.
- Run multiple workload scenarios and disclose whether results come from simulation or real infrastructure.

7. Challenges, Limitations, and Future Scope

7.1 Dynamic demand and uncertain runtimes

Runtime estimates and resource demand can be wrong. An instance that seems cheapest using a point estimate may become expensive if a job runs longer than expected, while a very small instance may miss a deadline. A real system should use historical measurements where available, update estimates, and keep a safety margin for important workloads.

7.2 Multi-resource constraints

CPU and memory are not interchangeable. Some jobs are CPU-bound, while others depend on memory bandwidth, disk throughput, or network performance. A cost model that considers only CPU cores may systematically select unsuitable machines. Eligibility should be based on the resource dimensions that matter to the application.

7.3 Shared capacity, migration, and failures

When several workloads are placed on the same host, their combined demand must fit the available capacity over time. Moving a running VM can consume network bandwidth, delay the application, and create additional expense. Over-consolidation can also increase the risk that a host failure affects many workloads. Placement rules should account for these interactions when the system requires them.

7.4 Fairness, security, and sustainability

Cost optimization should not consistently push low-priority or less profitable users into poor service. Security rules may prevent workloads from sharing a host, and compliance requirements may restrict where data can be processed. Energy or carbon goals can also conflict with minimum cost: the cheapest region or time may not have the lowest emissions. These objectives should be stated explicitly rather than hidden inside an unexplained score [3,5,6].

7.5 Future directions

A useful next step would be a small CloudSim-based study using a documented workload trace, a fixed price model, and repeatable baselines. The evaluation could compare cheapest-fit, best-fit, and an exact optimizer on smaller instances, then measure the cost of increasing workload variability. Another direction is hybrid allocation: use a prediction model to estimate demand, apply a fast heuristic to most jobs, and reserve optimization for high-impact decisions. Any such system should retain a human-readable explanation for why each allocation was selected.

8. Conclusion

Cloud resource allocation is not simply a question of selecting the lowest hourly price. The decision must match resource capacity to workload demand while respecting deadlines, service quality, security, and operational limits. A cheapest-feasible heuristic provides a transparent starting point and can avoid obvious over-provisioning in a simplified setting. Its simplicity is also its main limitation: local choices do not guarantee the best global placement, especially when workloads share hosts and demand changes over time.

The worked example demonstrates the arithmetic, not a measured performance result. A credible implementation should be evaluated against clear baselines using the same workloads and should report cost together with service and resource metrics. The central conclusion from the reviewed literature is that algorithm choice depends on the scale of the problem, the quality of available data, the time available to make decisions, and the trade-offs the system is expected to satisfy.

References

[1] Mann, Z. A. (2015). Allocation of virtual machines in cloud data centers - A survey of problem models and optimization algorithms. ACM Computing Surveys, 48(1), Article 11, 1-34. https://doi.org/10.1145/2797211

[2] Zhang, J., Huang, H., & Wang, X. (2016). Resource provision algorithms in cloud computing: A survey. Journal of Network and Computer Applications, 64, 23-42. https://doi.org/10.1016/j.jnca.2015.12.018

[3] Vakili Fard, M., Sahafi, A., Rahmani, A. M., & Mashhadi, P. S. (2020). Resource allocation mechanisms in cloud computing: A systematic literature review. IET Software, 14(6), 638-653. https://doi.org/10.1049/iet-sen.2019.0338

[4] Aron, R., & Abraham, A. (2022). Resource scheduling methods for cloud computing environment: The role of meta-heuristics and artificial intelligence. Engineering Applications of Artificial Intelligence, 116, 105345. https://doi.org/10.1016/j.engappai.2022.105345

[5] Madni, S. H. H., Faheem, M., Younas, M., Masum, M. H., Shah, S., et al. (2024). Critical review on resource scheduling in IaaS clouds: Taxonomy, issues, challenges, and future directions. The Journal of Engineering, 2024(8), e12420. https://doi.org/10.1049/tje2.12420

[6] Alam, S., Ramzan, M. A., Zubair, M., et al. (2026). AI-driven resource allocation in cloud computing: A systematic review revealing critical sustainability and evaluation gaps. Computing, 108, Article 67. https://doi.org/10.1007/s00607-026-01655-8

Source note

The paper summarizes and discusses the listed publications in its own words. Numerical values in Section 5 are hypothetical and are not derived from these sources. The 2026 review is cited for its reported review scope and methodological observations, not as evidence that the illustrative allocation in this paper would obtain the same results. Accessed online on 25 October 2026.

Submission checklist
Before submitting, enter your own name and enrollment number on the cover page, add your faculty member’s name, and read the worked example so you can explain its assumptions and calculation.